

ZETRA Flutter Development Guide

1. Architecture

Clean Architecture

We chose **Clean Architecture** because it provides clear separation of concerns, improves maintainability and testability, and enables the application to scale efficiently as new features and modules are added.

Project Structure

```
lib/
├── app/
│   ├── routes/
│   ├── themes/
│   └── localization/
├── core/
│   ├── api/
│   ├── network/
│   ├── storage/
│   ├── constants/
│   ├── widgets/
│   ├── services/
│   ├── errors/
│   └── extensions/
├── features/
│   ├── authentication/
│   ├── charging/
│   ├── station/
│   ├── wallet/
│   ├── history/
│   ├── notification/
│   └── profile/
```

└─ main.dart

Benefits

- Clear separation of responsibilities
- Easier maintenance and feature expansion
- Improved testability
- Better scalability for enterprise applications

2. State Management

Bloc / Cubit

Bloc remains one of the most widely adopted state management solutions in enterprise Flutter applications.

Why Bloc?

- Predictable state flow
- Explicit Event → State architecture
- Easier onboarding for developers
- Mature ecosystem and community support
- Proven in large-scale production applications

3. Network Layer

Dio

Dio is the recommended HTTP client for all API communications.

Features

- Request and Response Interceptors
- Authentication Handling
- Token Refresh Mechanism
- Request Cancellation
- Timeout Management
- Retry Support
- File Upload and Download Progress
- FormData Support

- Centralized Configuration
- Advanced Error Handling

Required Components

- ApiClient
- AuthInterceptor
- TokenInterceptor
- LoggerInterceptor

Flow

Request



Interceptor



Dio



Response Handler



Repository

4. Dependency Injection

GetIt + Injectable

Dependency Injection keeps the application modular, maintainable, and testable while preventing dependency management issues as the project grows.

Register Dependencies

- Repositories
- Data Sources
- Use Cases
- Cubits / Blocs
- Services

Standard

Avoid Direct Instantiation

✗ Incorrect

```
final api = ApiService();
```

✅ Correct

```
getIt<ApiService>();
```

5. Routing

GoRouter

GoRouter is the recommended routing solution for enterprise Flutter applications.

Features

- Deep Linking
- Nested Navigation
- Route Guards
- Authentication-Based Navigation
- Session Management
- Web Support

6. Design System

Create reusable design tokens to maintain visual consistency throughout the application.

Required Tokens

- AppColors
- AppSpacing
- AppRadius
- AppTypography
- AppIcons
- AppShadows

Example

```
AppColors.primary
```

```
AppSpacing.md
```

```
AppRadius.large
```

Standard

✗ Avoid Hardcoded Values

```
padding: EdgeInsets.all(17);
```

✓ Use Design Tokens

```
padding: EdgeInsets.all(AppSpacing.md);
```

7. Shared Components

All common UI components must be reusable and centralized.

Required Components

- PrimaryButton
- SecondaryButton
- AppTextField
- AppCard
- AppBottomSheet
- AppDialog
- AppLoader
- AppSnackbar
- AppEmptyState
- AppErrorState

Benefits

- Consistent UI/UX
- Faster development
- Easier maintenance

8. API Response Standardization

All API responses must follow a unified structure.

Create

- ApiResponse

- Success
- Failure
- Exception

Example

✓ Preferred

`Result<WalletModel>`

✗ Avoid

`dynamic`

`Map<String, dynamic>`

Benefits

- Type Safety
- Consistent Error Handling
- Cleaner Codebase

9. Error Handling

Implement centralized error handling across the application.

Handle

- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 500 Server Error
- Network Errors
- Timeout Errors
- Socket Exceptions

Rule

Always display meaningful and user-friendly error messages.

10. Logging & Monitoring

Sentry

Use Sentry for:

- Non-Fatal Error Tracking
- Performance Monitoring
- User Journey Analysis
- Production Diagnostics

Track Events

- Login Failed
- Payment Failed
- Charging Started
- Charging Stopped
- Session Interrupted
- QR Validation Failed

11. Local Database

Drift

Drift (formerly Moor) is the recommended local database solution.

Benefits

- Compile-Time Type Safety
- Reactive Queries
- Advanced Migrations
- SQLite Based
- Cross-Platform Support
- Excellent Documentation

Usage

Store:

- Charging History
- Cached Stations
- User Preferences
- Offline Data

12. Security

Security implementation is mandatory for all environments.

SSL Pinning

SSL Pinning ensures the application trusts only the configured server certificate or public key.

Benefits

- Prevents Man-in-the-Middle (MITM) attacks
- Prevents Certificate Spoofing
- Protects Sensitive User Data

Secure Storage

Use:

`flutter_secure_storage`

Store

- Access Token
- Refresh Token
- User Session
- Sensitive User Information

13. Maps Integration

Google Maps

Google Maps is the preferred mapping solution for ZETRA.

Benefits

- Excellent road and POI coverage
- Stable Flutter package support
- Familiar user experience
- Strong EV ecosystem support

Required Features

- Charger Markers
- Marker Clustering
- User Location Tracking
- Polylines
- Route Navigation
- Traffic Information
- Geocoding
- Places API Integration

14. QR Scanner Module

Mobile Scanner

Use:

`mobile_scanner`

Benefits

- Active Maintenance
- High Performance
- ML Kit Integration
- Apple Vision Integration
- Fast Scanning
- Lightweight Architecture

Features

- Real-Time QR Scanning
- Flash Control
- Zoom Support
- Custom Overlay UI
- Multiple Barcode Formats
- BLoC Compatibility
- Web Support

Flow

Scan QR



Validate QR



Fetch Charger Details



Start Charging Session

15. Real-Time Communication

WebSocket

Use WebSocket for all live charger communication.

Real-Time Data

- Charger Status
- Charging Session Updates
- State of Charge (SOC)
- Voltage
- Current
- Power Consumption
- Charging Progress

Rule

Avoid frequent API polling whenever WebSocket communication is available.

16. Testing Standards

Testing is mandatory for all critical business features.

Unit Tests

Cover:

- Repositories
- Use Cases
- Cubits / Blocs

- Services

Widget Tests

Cover:

- Login
- Wallet
- Charging
- Station Listing
- QR Scanner

Integration Tests

Cover:

- Complete Authentication Flow
- Complete Charging Flow
- Wallet Top-Up Flow
- Payment Flow

Minimum Target

- Critical Business Logic Coverage $\geq 80\%$

17. OTP Authentication

Recommended Solution

Firebase Authentication

+

Phone OTP

+

Backend JWT Token

Benefits

- Secure
- Scalable
- Easy Integration
- Android Support
- iOS Support
- Fraud Protection

18. Payment Gateway

India

- Razorpay

Malaysia

- iPay88

Global

- Stripe

Benefits

- Secure Transactions
- Wallet Top-Up Support
- Subscription Support
- Recurring Payments
- Enterprise Reliability

Enterprise Development Rules

Every feature must follow the architecture below:

UI Layer

↓

Cubit / Bloc

↓

Use Case

↓

Repository

↓

Data Source

↓

API / Database

Mandatory Rules

- ✓ No API Calls Inside Widgets

- ✓ No Business Logic Inside UI
- ✓ No Hardcoded Strings
- ✓ No Hardcoded Colors
- ✓ No Hardcoded Sizes
- ✓ Use Design Tokens
- ✓ Use Dependency Injection
- ✓ Every Feature Must Be Independent
- ✓ Every Feature Must Be Testable
- ✓ Follow Clean Architecture Principles
- ✓ Maintain Production-Level Code Quality

Project: ZETRA EV Charging Platform

Technology Stack: Flutter Enterprise Architecture

Development Standard: Production-Grade Enterprise Application